

Scientific Data Visualization

Final Project Report

Realtime VR Volume Renderer with SSS Support

Lumiverse

Andrew Tate and Alexandrew Christensen

April 16, 2026

Overview and Goals

Originally, we just wanted to create a volume renderer that functioned in VR to give the illusion that you were physically interacting with the volume. The idea was that the user would have an adjustable desk and floating screens that would adjust the transfer function, load in the volumes, and adjust shading. However, early on in the semester, a classmate mentioned how cool it would be to hold the volume up to a light source and see light pass through the volume. That stuck with us, so we also made it a goal to achieve some form of real-time subsurface scattering. If we had to summarize the goal into one sentence, it would be: *to make a real-time subsurface scattering-enabled, multi-use volume rendering software with Meta Quest VR support.*

Background and Related Work

What we looked into, in addition to what the class was already teaching us, was volume rendering, subsurface scattering, and we decided to use the Microfacet shade function instead of the older Blinn-Phong BRDF, which makes everything look more like plastic. Andrew quite confidently approached the Cook-Torrance model BRDF, as he had managed to implement it last semester in the Ray-Tracing class that we were both in. Ray tracing also helped us, as ray marching is really not that far off from ray casting; you simply have more intersections to worry about. Ultimately, the goal was to make what you were looking at feel a bit more organic, as we did much of our testing with the hand that was provided in class.

Referenced Papers

- **Shading model** — Walter et al. (2007), EGSR. “Microfacet Models for Refraction through Rough Surfaces.” This is the foundational GGX/Cook-Torrance paper covering the NDF, Smith geometry term, and Fresnel combination.
<https://www.graphics.cornell.edu/~bjw/microfacetbsdf.pdf>
- **Volume rendering** — Levoy (1988), IEEE CG&A. “Display of Surfaces from Volume Data.” The classic paper establishing ray casting through a scalar field with opacity

accumulation and gradient-based shading, which is essentially the core of the ray-march loop.

<https://graphics.stanford.edu/papers/volume-cga88/volume.pdf>

- **Subsurface Scattering** — Jensen et al. (2001), SIGGRAPH. “A Practical Model for Subsurface Light Transport.” This is the dipole model paper, which is the closed-form method our implementation departs from. For the Henyey-Greenstein phase function specifically, the source is a 1941 astrophysics paper by Henyey and Greenstein, most accessibly described in Pharr & Humphreys’ *Physically Based Rendering*.
<http://www.graphics.stanford.edu/papers/bssrdf/bssrdf.pdf>
- **Single and diffusion scattering for skin shaders** — Hery (2003), SIGGRAPH RenderMan course. Goes into the details of single and diffusion scattering, which closely aligns with our SSS implementation approach.

Project Description

We will start by giving a general verbal walkthrough of the software. The user starts in a room, decorated like a cozy apartment, with a desk and four floating screens surrounding the desk. Each of these screens is toggleable.

File Browser Panel

The leftmost screen displays a file directory where the user can put `.raw` and `.mhd` files. If the user selects an `.mhd` file, it will auto-populate dimensions and data size, but it also contains the option to set these settings manually. In addition, there are settings to downscale the data linearly (in the case that the data is simply too large to reasonably fit in the Quest 2’s 6 GB of total system RAM). There are also texture filtering options, offering point, bilinear, and trilinear texture options. Finally, there is a load button that will fade the screen to black until the volume is parsed and converted into a 3D texture.

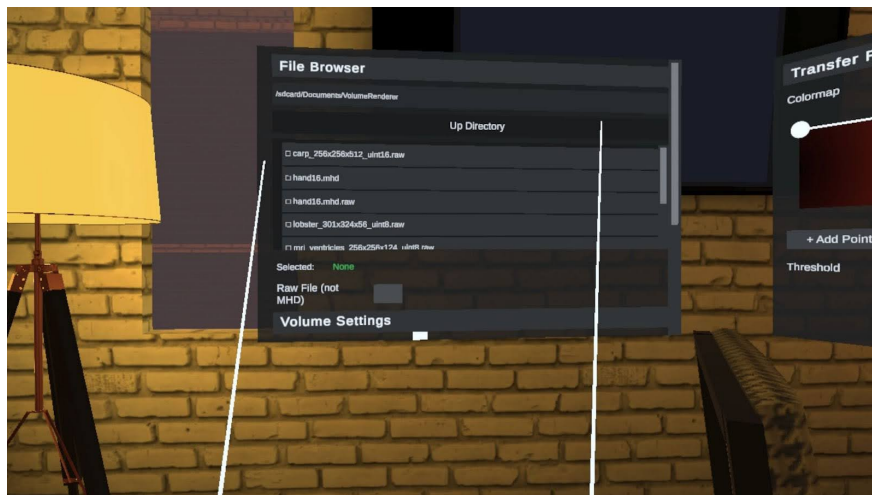


Figure 1: The file browser panel.

Transfer Function Panel

Screen two contains the actual transfer function with multiple preset colormap options. It defaults to a straight line from the top-left of the gradient to the top-right. It contains an option to add or remove a selected point. The user can point with the controller and drag the transfer function points to adjust the opacity of a given point on the gradient, and the graph will automatically adjust and sort the points, connecting them to visualize the transfer function. As soon as the user releases a point, the transfer function bakes to a texture and gets sent to the volume rendering shader. Finally, it also shows a cut-off “floor” value the user can set, which essentially sets all values of the gradient’s x -axis to the left of that value to an opacity of 0.

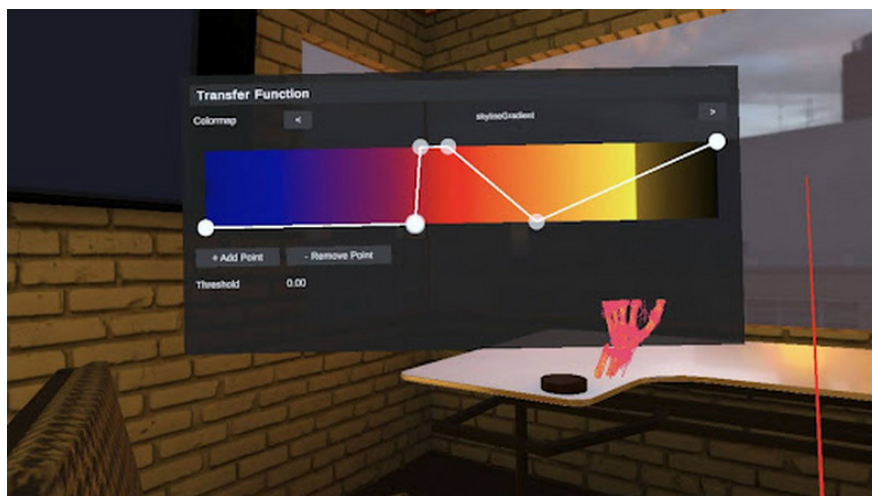


Figure 2: The transfer function panel.

Shading Settings Panel

Screen three contains our shade settings. It has options to toggle shading on and off, and settings for light intensity, metalness, roughness, and ambient light. All of these settings work in real-time and immediately update the volume’s visual features. Additionally, the light intensity is directly tied to a lightbulb hanging above the desk, which visually brightens and dims accordingly.



Figure 3: The shading settings panel.

Subsurface Scattering Panel

The user cannot use subsurface scattering unless shading is enabled. Our SSS panel contains settings for a coefficient/transmittance value, steps, spherical samples, spherical sample radius, and anisotropy. These settings are expanded upon in Section 4.



Figure 4: The Subsurface Scattering (SSS) panel.

The Table and Physical Interaction

The table has a handle on the side, allowing the user to raise and lower the desk that the volume sits upon. It also has two knobs: one adjusts the threshold value mentioned in the transfer function section, and the other adjusts the physical size of the volume. The goal of these knobs was to give the illusion of physical feedback and to allow micro-adjustments to comb through different densities of the volume. Most importantly, the user can physically

pick up the volume they are rendering, hold it up to the light, and view any arbitrary angle with the ease of holding any real-life object.



Figure 5: User adjusting the table with a shrunken hand model on top.

Datasets

We primarily used the hand dataset provided in assignment 3, but the software also preloads the present from the same assignment. Additionally, we preloaded volumes from <http://klacansky.com/open-scivis-datasets/>, notably the carp, tooth, and lobster datasets. Due to the nature of our project being a general-purpose renderer rather than a specific visualization, the dataset usage is relatively arbitrary, and any volume data in the `.raw` format can be used.

Research Questions

Our looming thought was that having a physical object you can hold up to a light and see a physically-inspired simulation of how light passes through it aids in understanding the general structure of the volume. While we have not conducted a full user study, we can confidently say that it does, and it offers a level of interactivity that software such as ParaView cannot match.

Several interesting directions emerged during the project:

- After observing the performance impact of SSS, we have been thinking about more efficient sampling strategies, such as low-discrepancy sequences like Halton, to pick better regions of the volume to compute light contribution without introducing aliasing.
- We are also curious whether precomputing all lighting in a lookup texture during a compute shader pass before the main volume rendering would be more efficient, since the headset renders two shader copies per eye and SSS is not dependent on camera position.

Implementation Details

This project grew quite large in scope, so we will focus on the parts most crucial to getting the volume renderer running on the platform. The project was built in Unity 2023, chosen for its integration with OpenXR and the XR Interaction Toolkit, allowing us to more easily target our VR platform. UI was implemented with Unity’s UI Toolkit, which handled VR controller input and raycasting to UI elements. We chose this specific version of Unity because it is the last version to support the Default Rendering Pipeline, as opposed to URP. URP no longer primarily supports HLSL as the default shading method, opting instead for a Shader Graph, which would have significantly complicated our workflow. Accordingly, the main bulk of our logic was written in HLSL.

Rendering Pipeline

The main pipeline of the renderer proceeds as follows:

1. **Volume parsing.** The raw file is parsed based on either the `.mhd` file or user-set parameters, and uploaded to the fragment shader as a 3D lookup table (LUT) of densities.
2. **Normal and density precomputation.** Normals are precalculated at load time by differencing one step forward and backward along each axis to produce a gradient vector perpendicular to the local isosurface. This is computed at a downsampled resolution and baked into an `RGBAHalf` texture, with density packed into the alpha channel. This is crucial to saving per-frame computation.
3. **Transfer function baking.** Rather than passing control points to the fragment shader and recomputing each frame, we compute each x -value on the gradient as a `float4` with interpolated opacity and bake this into a texture, which is updated only when the transfer function is modified.
4. **Ray-box intersection.** In the shader, the camera is transformed into the bounding box’s local space (from -0.5 to 0.5 on all axes). A ray-box intersection determines exactly where the ray enters and exits the bounding volume.
5. **Ray marching.** Any point within the bounding box is sampled as $\mathbf{p} = \mathbf{o} + t \cdot \hat{\mathbf{d}}$, where t increments by a fixed step size. The iso-value is sampled from the 3D texture and compared against the threshold; sub-threshold points are skipped.
6. **Shading.** Normals are looked up from the precomputed texture. Shading uses the Cook-Torrance microfacet BRDF with:
 - GGX for the normal distribution function (NDF),
 - Smith’s method for the geometry/shadowing term, and
 - Schlick’s approximation for the Fresnel term.

Environment map reflections are added via Unity’s reflection probe.

Subsurface Scattering

Traditional SSS simulates light entering a surface, scattering through the material, and exiting at a different point. This is typically solved with a closed-form dipole model (Jensen et al. 2001). The key insight is the distinction between *in-scattering* (light scattered from a different path towards the camera ray) and *out-scattering* (light scattered away from the camera before reaching it).

Our approach is a Monte Carlo approximation. For each sampled point in the volume, we do two things:

1. **Direct light march.** We march towards the light source by a given step count, calculating the amount of light that reached the point using the Beer-Lambert transmittance term:

$$T = \exp(-\sigma_a \cdot \sigma_s \cdot d)$$

where σ_a is the absorption coefficient, σ_s is the scattering coefficient, and d is the marched distance. This yields a direct light sample.

2. **Spherical sampling (pseudo-MIS).** Optionally, we sample a user-specified number of directions in a sphere of user-specified radius around the point and directly sample the light from each. These samples are averaged together, weighted by the PDF of sampling the sphere, and divided by the sample count.

The total contribution is finally weighted by the **Henyey-Greenstein phase function**, which controls whether light preferentially scatters towards or away from the camera based on the anisotropy parameter g :

$$p(\cos \theta) = \frac{1}{4\pi} \cdot \frac{1 - g^2}{(1 + g^2 - 2g \cos \theta)^{3/2}}$$

This is physically inspired but essentially a brute-force method rather than a traditional closed-form model. All parameters are user-controllable to ensure interactive performance on the Quest 2.

Scene Baking

To achieve decent visual quality in the room environment without incurring real-time cost, all global illumination, shadows, and reflections are baked using Unity’s environment baking features. This keeps the actual VR runtime cost focused on the volume rendering components.

What We Learned

This project gave us a much deeper understanding of how volume rendering works. The difference between adjusting settings in ParaView and building a renderer mostly from scratch significantly deepened our understanding. SSS was also new for both of us, so working through a real implementation was genuinely informative. The project also gave good practical experience in real-time graphics and performance optimization techniques for constrained hardware.

Differences From Proposal

We did not succeed at implementing some of our stretch goals, mainly those related to Google Drive API integration — which would have allowed loading `.raw` and `.mhd` files from the cloud directly into the headset — and saving presets to the cloud. Other than that, we adhered very closely to our original proposal.

Evaluation



Figure 6: The user holding a hand volume up to the light, demonstrating real-time subsurface scattering.

This far exceeded our expectations; we accomplished all core goals and nearly every stretch goal as well. There is always room for improvement; we can envision further optimizations as mentioned in the research questions above. The UI could also benefit from some quality-of-life additions, such as custom color maps and more physical dials. The renderer is also missing a 2D transfer function, which could help with some of the limitations of VR raycasting controls. Nevertheless, the presentation, performance, and usefulness of the project are far beyond what we thought would be achievable, and it is arguably one of the coolest things we have built.

Additional Comments

Given the VR and interactive nature of this project, a video demo is essentially required to appreciate it fully. A recording is linked here:

<https://youtu.be/hKsS2Zd7T9c>

The GitHub repository (containing a link to the APK for VR devices and some Unity-independent code) and the public Itch.io page are linked below:

<https://andrew-wilson.itch.io/lumiverse>

<https://github.com/tater-tot25/Lumiverse/tree/main>

Finally, while unrelated to Scientific Data Visualization, we went above and beyond and included a couch with a controller, where the user can take a break from volume rendering and play Snake on a TV in the room.



Figure 7: The user enjoying a game of Snake between tiresome volume rendering sessions.